Full Stack Assignment – 48

Assignment ID: FSD-48

Title: SLA-Based Ticket Aging and Breach Management System

Duration: 3 Hours

1. Objective

Design and develop an **SLA-Based Ticket Aging and Breach Management System** that tracks service-level agreements (SLAs) for tickets and flags breaches when response or resolution timelines are exceeded. The system must enforce SLA calculations accurately and ensure **data persistence using a database**.

2. User Roles

REQUESTER

- Register and log in to the system
- Create tickets with priority
- View ticket status and SLA timers

AGENT

- Register and log in to the system
- Respond to and resolve assigned tickets
- View SLA status for assigned tickets

ADMIN

- Register and log in to the system
- Define SLA rules by priority
- View SLA compliance and breach reports

3. Functional Requirements

Authentication & Authorization

- Implement user registration and login
- Use JWT-based authentication
- Protect all backend APIs using JWT
- Enforce role-based access control on APIs and frontend views

Ticket Management

- REQUESTER can create tickets with priority (e.g., Low, Medium, High)
- Tickets must be stored in the database
- AGENT can update ticket status (e.g., Open, In Progress, Resolved)
- Ticket updates must be persisted and retrievable

SLA Configuration

- ADMIN can define SLA rules per priority:
  - Response time limit
  - Resolution time limit
- SLA rules must be stored in the database

SLA Tracking and Aging

- System must calculate elapsed time since ticket creation and last response
- SLA timers must be computed using server time
- SLA status must be stored and updated in the database

Breach Detection

- Tickets must be marked as **BREACHED** if SLA limits are exceeded
- Breach status must persist and be visible to authorized roles

4. Business Rule (Critical)

Once an SLA breach is detected for a ticket, the breach status must be permanent and must not be cleared even if the ticket is later resolved.

5. Technical Constraints (Mandatory)

- All ticket, SLA, and breach data must persist in a database
- In-memory or hardcoded SLA calculations are not allowed
- Backend APIs must reflect actual database state
- JWT authentication and role validation are mandatory
- Use environment variables for sensitive configuration

6. Frontend Requirements

- Login and registration pages
- Role-based dashboards for REQUESTER, AGENT, and ADMIN
- Ticket listing with SLA timers and breach indicators
- SLA configuration and reporting views for ADMIN

UI design is not evaluated; correctness of SLA computation and breach detection is the priority.

7. Submission Requirements (Strict)

GitHub Repository

- Backend source code
- Frontend source code
- Proper folder structure

- Clear and meaningful commit history

Live Deployment

- Live frontend deployment link
- Live backend deployment link
- Backend must be connected to the database

README.md (Mandatory)

The README file must include:

- Project overview
- Tech stack used
- User roles and permissions
- API endpoints
- Database schema or collections
- Steps to run the project locally
- Live frontend and backend links

8. Evaluation Note

Evaluation focuses on **SLA calculation accuracy, breach immutability, database-backed timing logic, role enforcement, and end-to-end execution**.